

SOFTWARE REQUIREMENTS SPECIFICATION (SRS)

CampusCart

Student Marketplace Platform

Technology Stack: React + Vite | Node.js + Express | MongoDB Atlas

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) describes the functional and non-functional requirements of CampusCart — a web-based student marketplace platform. It is intended for developers, reviewers, and maintainers involved in building or extending the system.

1.2 Scope

CampusCart allows students to list second-hand items for sale, browse marketplace listings, send messages to other users, and report suspicious listings. An admin role manages marketplace quality by approving student signups and moderating listings.

1.3 Definitions and Abbreviations

Term	Meaning
Guest	Unauthenticated user who can browse listings
Student	Registered and approved user who can post and message
Admin	Privileged user who approves students and moderates listings
Listing / Product	An item posted for sale on the marketplace
Conversation	Chat thread between two users, optionally tied to a listing
Report	Complaint raised by a user against a listing
Approval Status	Student account state: pending, approved, or rejected
SRS	Software Requirements Specification
API	Application Programming Interface
UI	User Interface
SPA	Single Page Application
DFD	Data Flow Diagram

1.4 References

Project README, SYSTEM_DESIGN.md, Use Case Diagram, Sequence Diagrams (Workflow A, B, C), Class Diagram, Activity Diagram, DFD Level 0 and Level 1, MongoDB Atlas Documentation, Express.js Documentation, React + Vite Documentation.

1.5 Overall Description

CampusCart is a 3-tier web application consisting of a React + Vite frontend SPA, a Node.js + Express REST API backend, and a MongoDB Atlas persistence layer. The frontend calls backend APIs for authentication, product management, messaging, and admin actions. The backend enforces role-based authorization and persistence across domain collections.

2. Overall Description

2.1 Product Perspective

CampusCart is a standalone web-based application. The three tiers are:

- Frontend: React + Vite SPA served as static files
- Backend: Node.js + Express REST API running on port 3001
- Database: MongoDB Atlas cloud database

2.2 Product Functions

- Guest: Browse listings, view listing details, submit signup request
- Student: Login, post listings, report listings, send and manage messages
- Admin: Approve/reject student signups, delete listings, view dashboard metrics

2.3 User Classes

User Class	Description	Key Capabilities
Guest	Not logged in	Browse listings, view details, open login/signup
Student (Pending)	Signup submitted, awaiting admin action	Cannot log in until approved
Student (Approved)	Authenticated standard user	Post listing, browse, report, message, manage session
Admin	Authenticated privileged user	Dashboard, approve/reject students, delete listings, view metrics

2.4 Constraints

- Internet connection required for all operations.
- Students cannot log in until their signup request is approved by an admin.
- Marketplace transactions and payments are outside the system scope.
- Product image payloads are base64-encoded and bounded by API body-size limits.
- Only one conversation is permitted between any two users.

2.5 Assumptions and Dependencies

- Email addresses are unique per user account.
- The system depends on MongoDB Atlas being continuously accessible.
- Role-based access is enforced at both API and frontend route levels.
- The system runs on modern browsers (Chrome, Firefox, Safari) on campus network conditions.

2.6 Future Enhancements

- Mobile application (iOS / Android).
- In-app payment gateway integration for on-platform transactions.
- Email notification system for listing updates and message alerts.
- AI-powered listing recommendation engine.

3. Specific Requirements

3.1 Interface Requirements

3.1.1 External Interface

Web browser interface supporting Chrome, Firefox, Edge, and Safari. Responsive UI for desktop, tablet, and mobile devices.

3.1.2 Hardware Interface

Client device (PC, Laptop, Tablet, or Mobile) with internet access. Server with Node.js 18+ runtime hosting the backend API.

3.1.3 Software Interface

Frontend	React 18 + Vite, Tailwind CSS
Backend	Node.js 18+, Express.js
Database	MongoDB Atlas (cloud-hosted)
Authentication	bcrypt password hashing, session-based role context
API Protocol	REST over HTTP/HTTPS
Build Tool	Vite (frontend), npm (backend)

3.1.4 Communication Interface

- HTTP/HTTPS protocol for all client-server communication.

- JSON payloads for all API requests and responses.
- CORS configured at API level; origin controlled via environment variable.

3.2 Functional Requirements

3.2.1 Functional Requirements Table

ID	Requirement
FR-01	System shall allow guests to browse available listings.
FR-02	System shall allow users to view listing details by listing ID.
FR-03	System shall allow students to submit signup requests with name, email, and password.
FR-04	System shall reject login for students whose approval status is not approved.
FR-05	System shall authenticate users using email/password and return user role/profile on success.
FR-06	System shall allow approved students and admins to create listings with required fields (title, description, category, price, location, sellerEmail).
FR-07	System shall validate that listing price is a positive numeric value.
FR-08	System shall allow only admins to delete listings.
FR-09	System shall allow authenticated users to report listings with a reason and optional details.
FR-10	System shall prevent a user from reporting their own listing.
FR-11	System shall prevent duplicate pending reports for the same reporter and listing pair.
FR-12	System shall allow users to fetch their conversation list and messages.
FR-13	System shall allow users to send messages in conversations where they are participants.
FR-14	System shall allow conversation deletion only by a participant of that conversation.
FR-15	System shall allow admins to view pending student signup requests.
FR-16	System shall allow admins to approve student signup requests.
FR-17	System shall allow admins to reject student signup requests.
FR-18	System shall expose a health endpoint for API and database connectivity checks.

3.2.2 Use Case Specifications

Case 1

Use Case ID	UC-01
Use Case Name	Browse Listings

Actors	Guest, Student, Admin
Description	Any user can view available marketplace listings without logging in.
Preconditions	API and database are reachable.
Postconditions	Listings are shown, or an empty-state message is displayed.
Normal Flow	1. User opens marketplace. 2. Frontend requests product list. 3. Backend returns listings. 4. UI renders product cards.
Alternate Flows	If no listings exist, UI shows "No products found". If filters applied, filtered results are returned.
Exceptions	API timeout or failure: Frontend shows an error/fallback state.
Priority	High
Business Rules	FR-01
Assumptions	Guest access to read-only listing data is permitted.

Case 2

Use Case ID	UC-02
Use Case Name	View Listing Details
Actors	Guest, Student, Admin
Description	User opens a specific listing to see its full details.
Preconditions	A valid listing ID is present in the route.
Postconditions	Listing details are rendered, or a not-found message is shown.
Normal Flow	1. User clicks a listing. 2. Frontend requests listing by ID. 3. Backend queries product. 4. UI shows detail page.
Alternate Flows	If optional fields are missing, defaults are displayed.
Exceptions	Invalid ID returns 400. Listing not found returns 404.
Priority	High
Business Rules	FR-02
Assumptions	Product IDs are unique and stable.

Case 3

Use Case ID	UC-03
Use Case Name	Submit Student Signup
Actors	Guest
Description	A guest submits a registration request which an admin must approve before login is allowed.

Preconditions	Name, email, and password are provided; email is not already in approved or admin status.
Postconditions	Student record is created with approvalStatus = pending.
Normal Flow	1. Guest opens signup form. 2. Fills name, email, password. 3. Backend validates and stores request. 4. Success message is returned.
Alternate Flows	A previously rejected student re-applying resets their status to pending.
Exceptions	Missing fields returns 400. Already approved/pending email returns 409.
Priority	High
Business Rules	FR-03, FR-04
Assumptions	Admin approval workflow is active before student login.

Case 4

Use Case ID	UC-04
Use Case Name	Login
Actors	Student, Admin
Description	Registered user authenticates with email and password.
Preconditions	Account exists; student approval status is approved.
Postconditions	Authenticated user profile is returned; session context is established.
Normal Flow	1. User enters credentials. 2. Backend validates password. 3. Role-based redirect occurs (Admin to dashboard, Student to marketplace).
Alternate Flows	Admin login redirects to admin dashboard.
Exceptions	Invalid credentials returns 401. Pending/rejected student returns 403.
Priority	Critical
Business Rules	FR-04, FR-05
Assumptions	Password hashes and role data are available in the database.

Case 5

Use Case ID	UC-05
Use Case Name	Post Listing
Actors	Student, Admin
Description	Authenticated user creates a new marketplace listing.
Preconditions	User is authenticated; required fields are present; price is a positive number.
Postconditions	New product record is created with status = available.

Normal Flow	1. User fills listing form (title, description, category, price, location). 2. Frontend submits payload. 3. Backend validates and assigns a sequence ID. 4. Product is saved and returned.
Alternate Flows	If image is omitted, a default image is used.
Exceptions	Missing fields or invalid price returns 400.
Priority	High
Business Rules	FR-06, FR-07
Assumptions	Seller email maps to an existing approved user account.

Case 6

Use Case ID	UC-06
Use Case Name	Report Listing
Actors	Student, Admin
Description	Authenticated user submits a complaint against a listing.
Preconditions	Reporter is authenticated; listing exists; reporter is not the listing seller; reason is valid.
Postconditions	Report is stored with status = pending.
Normal Flow	1. User opens report form. 2. Selects a reason, optionally adds details. 3. Backend validates and saves the report.
Alternate Flows	Details text is optional; report is stored without it.
Exceptions	Self-report attempt returns 400. Duplicate pending report returns 409.
Priority	Medium
Business Rules	FR-09, FR-10, FR-11
Assumptions	Report reasons are restricted to a configured set of enum values.

Case 7

Use Case ID	UC-07
Use Case Name	View Conversations
Actors	Student
Description	Student loads their conversation list and message history.
Preconditions	Student is authenticated with a valid user ID.
Postconditions	Conversations and messages are displayed sorted by latest activity.
Normal Flow	1. Student opens Messages page. 2. Frontend calls conversations endpoint. 3. Backend returns threads and messages. 4. UI renders the chat list.

Alternate Flows	No conversations: UI shows "No conversations found".
Exceptions	Missing/invalid userId returns 400. Query error returns 500.
Priority	High
Business Rules	FR-12
Assumptions	Participant IDs in conversations match valid users.

Case 8

Use Case ID	UC-08
Use Case Name	Send Message
Actors	Student
Description	Student sends a message inside a conversation.
Preconditions	Sender and receiver are valid, different users; message text is non-empty.
Postconditions	Message is stored; conversation lastMessageAt is updated.
Normal Flow	1. Student types and sends message. 2. Backend validates participants. 3. If new conversation, backend creates one automatically. 4. Message is saved and returned.
Alternate Flows	If conversation ID is omitted, backend auto-creates/finds a 2-party conversation.
Exceptions	Unauthorized participant returns 403. Missing fields return 400.
Priority	Critical
Business Rules	FR-13
Assumptions	Payload includes enough context to resolve participants.

Case 9

Use Case ID	UC-09
Use Case Name	Delete Conversation
Actors	Student
Description	Student deletes a conversation they are part of.
Preconditions	Conversation exists; requester is a participant.
Postconditions	Conversation and all related messages are removed.
Normal Flow	1. Student selects delete. 2. Frontend sends delete request. 3. Backend verifies participation. 4. Messages and conversation are removed.
Alternate Flows	Conversation already deleted: returns not-found response.
Exceptions	Not a participant returns 403. Conversation not found returns 404.

Priority	Medium
Business Rules	FR-14
Assumptions	Hard-delete is acceptable for chat lifecycle.

Case 10

Use Case ID	UC-10
Use Case Name	View Pending Requests
Actors	Admin
Description	Admin views student accounts waiting for approval.
Preconditions	Admin is authenticated.
Postconditions	List of pending student requests is returned and displayed.
Normal Flow	1. Admin opens moderation section. 2. Frontend requests pending students. 3. Backend validates admin role and returns records.
Alternate Flows	No pending requests: empty list shown.
Exceptions	Non-admin requester returns 403.
Priority	High
Business Rules	FR-15
Assumptions	Student approval statuses are accurate in the database.

Case 11

Use Case ID	UC-11
Use Case Name	Approve Student Request
Actors	Admin
Description	Admin approves a pending student signup request.
Preconditions	Admin is authorized; target student request exists.
Postconditions	Student status changes to approved.
Normal Flow	1. Admin selects a pending student. 2. Confirms approve action. 3. Backend updates approval status. 4. Success response returned.
Alternate Flows	Already approved/rejected: idempotent or conflict handling applies.
Exceptions	Student not found returns 404. Unauthorized user returns 403.
Priority	High
Business Rules	FR-16
Assumptions	Approval actor (approvedBy) is stored for audit purposes.

Case 12

Use Case ID	UC-12
Use Case Name	Reject Student Request
Actors	Admin
Description	Admin rejects a pending student signup request.
Preconditions	Admin is authorized; target student exists with pending status.
Postconditions	Student status changes to rejected.
Normal Flow	1. Admin selects pending request. 2. Triggers reject action. 3. Backend updates status to rejected.
Alternate Flows	Rejected student may re-submit through UC-03.
Exceptions	Unauthorized user returns 403. Student not found returns 404.
Priority	High
Business Rules	FR-17
Assumptions	Rejection does not permanently block future re-submission.

Case 13

Use Case ID	UC-13
Use Case Name	Delete Listing (Moderation)
Actors	Admin
Description	Admin removes a listing from the marketplace.
Preconditions	Admin is authenticated; listing exists.
Postconditions	Listing is deleted and no longer visible.
Normal Flow	1. Admin selects listing. 2. Sends delete request with adminId. 3. Backend validates admin role. 4. Listing is deleted.
Alternate Flows	Listing already removed: returns not-found response.
Exceptions	Non-admin requester returns 403. Listing not found returns 404.
Priority	High
Business Rules	FR-08
Assumptions	Hard delete is acceptable for moderation policy.

Case 14

Use Case ID	UC-14
--------------------	-------

Use Case Name	View Admin Dashboard
Actors	Admin
Description	Admin views marketplace metrics, users, and listing management tools.
Preconditions	Admin is authenticated; API is available.
Postconditions	Dashboard panels and tables are populated with current data.
Normal Flow	1. Admin navigates to dashboard. 2. Frontend loads stats, users, and listings. 3. Backend validates role. 4. UI displays metrics and moderation controls.
Alternate Flows	Partial data load: available sections render while failed sections show fallback state.
Exceptions	Unauthorized access returns 403.
Priority	High
Business Rules	FR-15, FR-16, FR-17, FR-08
Assumptions	Dashboard data is derived from near real-time API reads.

3.2.3 Data Dictionary (Analysis Classes)

Collection / Class	Attributes
User	id (sequence), name, email, passwordHash, role (student / admin), approvalStatus (pending / approved / rejected), approvedBy, createdAt
Product	id (sequence), title, description, category, price, location, sellerEmail, image (base64), status (available / sold), createdAt
Report	id (sequence), listingId, reporterId, reason (enum), details, status (pending / resolved), createdAt
Conversation	id (sequence), participants [userId], productId (optional), lastMessageAt, createdAt
Message	id (sequence), conversationId, senderId, text, createdAt
CustomSequence	name (entity type), seq (current counter value)

3.3 Non-Functional Requirements

ID	Category	Requirement
NFR-01	Performance	95% of read API requests shall complete within 2 seconds under normal campus load.
NFR-02	Availability	API health endpoint shall indicate degraded state when database connection is unavailable.

NFR-03	Security	Passwords shall be stored as bcrypt hashes; plaintext storage is prohibited.
NFR-04	Security	Role-restricted operations (approve/reject/delete) must deny unauthorized users with 403 responses.
NFR-05	Reliability	Input validation errors shall return clear 4xx responses without crashing the service.
NFR-06	Scalability	Data model and APIs shall support growth in users, listings, and messages without schema redesign.
NFR-07	Maintainability	Backend shall be modularized by route, service, and model concerns for easier updates.
NFR-08	Usability	Frontend shall provide clear success/failure feedback for auth, listing, report, and messaging actions.
NFR-09	Portability	System shall run on Node.js 18+ and modern Chromium, Firefox, and Safari browsers.
NFR-10	Data Integrity	Numeric IDs generated via sequence counters shall remain unique per entity type.

3.4 Database Requirements

The system uses MongoDB Atlas as its NoSQL persistence layer. Collections: users, products, reports, conversations, messages, customSequence. The customSequence collection acts as a counter service ensuring unique sequential numeric IDs per entity type. No strict foreign key constraints exist in MongoDB; referential integrity is enforced at the application layer.

3.5 Design Constraints

- Decoupled architecture: React SPA frontend, Express REST API backend.
- Role-based access control (RBAC) enforced at both API middleware and frontend route guard levels.
- All financial or payment operations are out of scope for this system.
- Product images are stored as base64 strings; payload size is bounded by the API body-size limit.
- Students must complete admin-approved signup before they can log in or post listings.

4. Software System Attributes

4.1 Reliability

The system returns clear and consistent error responses (4xx, 5xx) for all failure scenarios. Input validation errors do not crash the server. MongoDB write operations are handled with error boundaries to prevent data corruption.

4.2 Availability

The application is designed for 24/7 availability with no scheduled downtime. The `/api/health` endpoint checks both API and database connectivity, enabling monitoring and automated alerting.

4.3 Security

- Passwords stored as bcrypt hashes — no plaintext storage.
- Role-based access: admin-only endpoints reject unauthenticated or non-admin callers with 403.
- CORS configured via environment variable to restrict allowed origins.
- Input validation on all request payloads to prevent injection attacks.

4.4 Maintainability

The backend is organized into modular concerns: routes, services, and models (`app.js`, `routes/`, `middleware/`). Each domain (products, users, admin, messages, auth) has its own route and logic files, making it straightforward to update or extend individual features.

4.5 Portability

The system is compatible with Windows, Linux, and macOS developer environments via Node.js 18+. The frontend build output is static HTML/JS/CSS deployable on any static hosting service. The backend can be containerized with Docker and deployed to cloud platforms such as AWS, Heroku, or Render.

5. Supporting Information

The supporting materials for this SRS include:

- Project README — setup instructions and API endpoint reference.
- SYSTEM_DESIGN.md — full use case specifications, DFD diagrams, and class diagrams.
- Use Case Diagram (AGILE METHODOLOGIES/USECASE DIAGRAM) — visual use case model.
- Sequence Diagrams — Workflow A (Signup/Approval/Login), Workflow B (Create Listing), Workflow C (Messaging).
- Class Diagram — data model and relationships.
- Activity Diagram — marketplace interaction activity flow.
- DFD Level 0 & Level 1 — system context and internal data flows.
- DATABASE_SETUP.md and IMPORT_DATABASE.md — database initialization guides.